

SAFE CHARGING CHIP FOR MOBILE PHONES

Presentation + Viva Survival Guide

Architecture: ESP32 for sensing/display + PYNQ-Z2 / Zynq-7000 FPGA for safety decision logic.

Project goal: monitor charging voltage, current and temperature and disconnect the charging path during a dangerous condition.

This guide is based on your supplied synopsis, current ESP32 prototype connection document, Arduino code, Vivado Verilog and FPGA explanation video. It separates what is verified from what is still an integration step.

1. THE WHOLE PROJECT IN 30 SECONDS

Memorize this first. If you get nervous tomorrow, come back to this.

“Our project is a Safe Charging Chip for mobile phones. The ESP32 handles sensing and display, while the PYNQ-Z2 FPGA acts as an independent hardware safety layer. We monitor voltage, current and temperature. The safety logic classifies the condition as SAFE, WARNING or FAULT. If a dangerous condition is detected, the charging path is disconnected through the relay and the user gets a red LED and buzzer indication.”

Signal flow: Voltage / Current / Temperature → ESP32 → UART → FPGA → Safety FSM → Relay + LEDs + Buzzer

Your synopsis explicitly describes this two-stage architecture, with ESP32 sensing/display and FPGA safety decision/cut-off.

Important status: the current verified prototype is the ESP32 stage using potentiometers as fault simulators. The synopsis marks full FPGA integration and real-sensor replacement as later implementation steps.

2. WHAT YOU SHOULD EXPLAIN

Problem: charging can become unsafe because of excessive voltage, current or temperature.

ESP32: reads sensors, handles LCD/display and local prototype logic.

UART: transfers measurements from ESP32 toward the FPGA.

FPGA: receives the data, parses it, compares it with thresholds and runs the safety FSM.

Relay: physically connects/disconnects the charging path.

Indicators: green = safe, yellow = warning, red + buzzer = fault.

Current prototype pins: voltage sensor GPIO32, ACS712 GPIO35, DS18B20 GPIO19, pots GPIO33/36/39, LCD SDA/SCL GPIO21/22, relay GPIO26, buzzer GPIO25, LEDs GPIO2/4/15.

LCD: I2C address 0x27.

3. FPGA EXPLANATION

If they ask “Why FPGA?”, answer this:

“The ESP32 is useful for sensing and display, but the FPGA gives us an independent hardware safety layer. Voltage, current and temperature checks can be implemented as dedicated hardware and evaluated in parallel with deterministic timing, without depending on an operating-system scheduler or application software loop.”

Conceptual comparison:

ESP32/software: read → check voltage → check current → check temperature → decide.

FPGA: separate hardware comparison paths for voltage/current/temperature → safety decision hardware → state machine → protection output.

VLSI angle: your Verilog describes digital hardware/IP that could later be mapped to a CPLD or ASIC. The current implementation is on FPGA, not a fabricated ASIC.

Do not claim a measured response time unless you actually measured it. Your synopsis lists fast cutoff as a validation target, not proof of a completed measurement.

4. YOUR SAFETY FSM

IDLE: waiting for data.

MONITOR: normal monitoring; checks warning/fault conditions.

WARNING: approaching unsafe region; charging is still allowed in the current prototype logic.

FAULT: dangerous condition; cutoff is activated.

Current Verilog fault limits: voltage > 5.5 V, current > 7.5 A, temperature > 40 °C, ESP fault F=1, or sensor fault S=0.

Current Verilog warning limits: voltage > 5.0 V, current > 6.5 A, temperature > 32 °C.

Anomaly checks: voltage change > 500 mV, current change > 1000 mA, or temperature rise > 1 °C between consecutive samples can raise warning logic.

Strong line: “We use both absolute thresholds and rate-of-change checks, so we can detect a dangerous value as well as a rapid abnormal change.”

Important: these threshold numbers are your current prototype/Verilog settings; they are not universal mobile-phone charging limits.

5. 3–5 MINUTE PRESENTATION SCRIPT

Opening: “Good morning. Our project is Safe Charging Chip for Mobile Phones. The objective is to provide an independent hardware-level protection layer during charging by monitoring voltage, current and temperature and disconnecting the charging path when an unsafe condition is detected.”

Problem: “Charging faults can involve over-voltage, over-current or excessive temperature. Our solution separates sensing from the safety decision. The ESP32 handles sensing and display, while the PYNQ-Z2 FPGA handles the safety decision logic.”

Architecture: “The sensors feed the ESP32. The ESP32 processes the readings and sends data to the FPGA through UART. The FPGA receives the data, compares it with safety thresholds and runs a state machine. The resulting state controls the protection outputs.”

Why FPGA: “We selected FPGA because the safety comparisons can be implemented as dedicated hardware and evaluated in parallel with deterministic timing. This provides an independent safety path.”

Demo: “The potentiometers simulate abnormal voltage, current and temperature conditions. Safe gives green, warning gives yellow, and fault gives red plus buzzer and disconnects the relay.”

Close: “The current prototype validates the ESP32 sensing/display and fault-indication stage. The next integration steps are FPGA bring-up, UART validation, real-sensor calibration and complete low-voltage dummy-load testing.”

6. MOST LIKELY VIVA QUESTIONS

Q: What is the objective?

Say: To provide an independent hardware safety layer for charging that monitors voltage, current and temperature and disconnects the charging path during a dangerous condition.

Q: Why ESP32?

Say: It is convenient for sensor interfacing, ADC reading, LCD control and UART communication.

Q: Why FPGA?

Say: For deterministic, parallel hardware safety logic that is independent of the ESP32 application software.

Q: What is UART?

Say: A serial communication method. Your current link is configured for 115200 baud, 8 data bits, no parity and 1 stop bit.

Q: What does the relay do?

Say: It controls the charging path and disconnects it during a fault.

Q: Why use three states?

Say: SAFE allows normal operation, WARNING gives early indication, and FAULT performs the protection action.

Q: Why use a potentiometer?

Say: It safely simulates changing sensor conditions so faults can be demonstrated without deliberately stressing a battery.

Q: Why not test on a real phone battery?

Say: Because intentionally overcharging or overheating a real lithium-ion battery is unsafe. The project is intended to use a dummy/low-voltage test setup.

Q: What does the LCD do?

Say: It provides local display of measured values and system status.

Q: What is the VLSI relevance?

Say: The safety decision is described as digital hardware in Verilog and implemented on FPGA fabric; the same IP concept can later be mapped toward CPLD/ASIC hardware.

7. TRICKY QUESTIONS — DON'T GET CAUGHT

Q: Is this actually a chip?

Say: The present prototype uses a PYNQ-Z2 FPGA board, not a fabricated ASIC. “Chip” describes the intended hardware safety IP architecture.

Q: Did you fabricate the VLSI chip?

Say: No. We implemented the digital safety logic as FPGA hardware. ASIC fabrication is future scope.

Q: Is the whole FPGA system already fully validated?

Say: No. The ESP32 prototype stage is the verified portion. The FPGA integration is the next stage of the project.

Q: Are the sensor values fully calibrated?

Say: Not yet. The current prototype documentation notes that the ACS712 and 0–25 V sensor still require calibration.

Q: Why not let the ESP32 control everything?

Say: The project goal is an independent hardware safety layer. The FPGA provides a separate deterministic decision path.

Q: What is the biggest limitation?

Say: Full FPGA integration, calibrated real-sensor validation and complete end-to-end low-voltage testing are still integration steps.

Q: What happens if the UART link fails?

Say: In the planned target architecture, a watchdog would treat missing valid frames as a fault. Your current Verilog does not yet implement that watchdog.

Q: Does your current UART use the final 5-byte protocol?

Say: No. Your current Arduino and Verilog use an ASCII text format. The 0xA5 + XOR-checksum 5-byte format is described as planned in the synopsis.

Q: Does your current FPGA calculate power?

Say: Yes, the Verilog calculates power internally from voltage $\times$ current, but the current safety FSM does not use power as a fault condition.

8. CODE-SPECIFIC QUESTIONS

Q: Why is BAUD_DIV about 1085?

Say: Because 125 MHz / 115200 baud is approximately 1085 clock cycles per UART bit.

Q: What does V=5000 mean?

Say: 5000 mV, or 5.0 V.

Q: What does T=3200 mean?

Say: 3200 centi-degrees, or 32.0 °C.

Q: What does F=1 mean?

Say: The ESP32 is reporting a fault.

Q: What does S=0 mean?

Say: The ESP32 is reporting a sensor fault.

Q: What is voltage_spike?

Say: It becomes active when the difference between consecutive voltage readings exceeds the configured 500 mV limit.

Q: What is current_surge?

Say: It becomes active when the difference between consecutive current readings exceeds 1000 mA.

Q: What is rapid_temperature_rise?

Say: It becomes active when temperature rises by more than 1 °C between consecutive samples.

Q: What is a state machine?

Say: A digital control structure with defined states and rules for moving between those states.

Q: Why calculate power in the FPGA?

Say: It gives the system an internal power estimate. In the present code it is calculated but not yet used as a safety decision.

9. WHAT NOT TO CLAIM TOMORROW

This is the most important honesty page.

- Do not say the FPGA stage is fully validated if you have not demonstrated it.
- Do not claim a measured millisecond cutoff time unless you measured it.
- Do not claim the current UART is the final 5-byte 0xA5/checksum protocol; your current code uses ASCII text.
- Do not claim the current temperature sensor is LM35; your working prototype uses the DS18B20 replacement.
- Do not claim the ACS712 and voltage readings are fully calibrated; your current connection document says calibration is still required.

If challenged: “Currently we have verified the ESP32 prototype stage. The FPGA safety layer is the integration stage we are validating next. We have kept the architecture modular so each stage can be tested independently.”

10. TOMORROW'S DEMO ORDER

1. **SAFE:** Put all simulated fault inputs low/safe. Show green indication and normal LCD.

2. **WARNING:** Increase one simulated parameter into the warning range. Explain that charging remains enabled but the system warns the user.

3. **FAULT:** Increase the parameter into the fault range. Show red LED, buzzer and relay cutoff.

4. **RESET:** Return the condition to a safe region and demonstrate your reset behavior if the current firmware permits it.

Your current ESP32 prototype documentation defines 0–49% SAFE, 50–79% WARNING and 80–100% FAULT for the potentiometer simulation.

Safety: Use only an isolated low-voltage DC test source. Never connect 230 V AC mains to the breadboard, ESP32, sensors, LCD or GPIOs.

11. LAST-MINUTE CHEAT SHEET

Question	One-line answer
Project?	Independent hardware safety layer for mobile charging.
ESP32?	Sensing, display and communication.
FPGA?	Independent, deterministic, parallel safety decision logic.
Inputs?	Voltage, current and temperature.
Communication?	UART, current code at 115200 baud.
SAFE?	Charging allowed; green indication.
WARNING?	Approaching unsafe range; charging remains allowed.
FAULT?	Red + buzzer; charging path disconnected.
Why pots?	Safe fault simulation.
Why FPGA?	Parallel hardware safety independent of software loop.
VLSI angle?	Verilog digital hardware/IP implemented on FPGA; ASIC/CPLD can be future scope.
Current limitation?	Full FPGA integration and calibrated real-sensor validation remain to be completed.

FINAL RULE: If you forget a technical detail, don't bluff. Explain what is verified, what is planned, and why the architecture is split into independent stages.

You do not need to memorize the code. You need to know the flow: **INPUT → PROCESS → STATE → PROTECTION.**